

GitNexus

Investigación de la Aplicación GitHub

- **Juan Sebastian Perafan Bohorquez**
 - **Johan Alejandro Bernal**
- **Carlos Alberto Caro Quezada**
 - **Cristian Bayona Alzate**
 - **Daniel Castillo Mora**
 - **Santiago Salgado**

Mayo 2026

1. Que es GitHub

GitHub es una plataforma web basada en Git, el sistema de control de versiones distribuido de código abierto más utilizado en el mundo. Permite a equipos de desarrollo almacenar, versionar, colaborar y gestionar código fuente desde cualquier lugar, a través de un navegador web o integraciones con IDEs y herramientas externas.

En 2026, GitHub cuenta con más de 100 millones de desarrolladores registrados y más de 420 millones de repositorios, convirtiéndose en el ecosistema más grande del mundo para software open source y privado. Es propiedad de Microsoft desde 2018 y se ha expandido más allá del control de versiones para convertirse en una plataforma integral de desarrollo de software.

Para el proyecto, la plataforma cumple dos roles fundamentales:

- Repositorio del proyecto: donde el equipo de trabajo guarda y versiona todos los artefactos de desarrollo.
- Fuente de datos: la API de GitHub expone información sobre repositorios, commits, pull requests, issues y más, que el backend consumirá para calcular y reportar KPIs.

2. Funcionalidades Principales

2.1 Repositorios y Control de Versiones

El núcleo de GitHub es el repositorio: un espacio de almacenamiento que contiene el historial completo de cambios de un proyecto. Basado en Git, permite:

- Crear ramas para desarrollar funcionalidades de forma aislada.
- Hacer commits para registrar cada cambio con autor, fecha y descripción.
- Usar tags para marcar versiones o releases del software.
- Ver el historial completo de modificaciones de cualquier archivo.

2.2 Pull Requests y Code Review

Los Pull Requests (PR) son el mecanismo principal de colaboración en GitHub. Permiten que un desarrollador proponga cambios al código base para que otros integrantes del equipo los revisen antes de fusionarlos. Incluyen:

- Comentarios inline sobre líneas específicas de código.
- Aprobaciones y solicitudes de cambio.
- Integración con CI/CD para ejecutar pruebas automáticas antes del merge.

2.3 Issues y Projects

GitHub Issues es el sistema de seguimiento de tareas y bugs integrado en la plataforma. GitHub Projects permite organizar los issues en tableros estilo Kanban o en vistas de tabla y roadmap. Para el equipo del proyecto, estas herramientas permiten:

- Asignar las tareas semanales a integrantes específicos.

- Rastrear el progreso de cada actividad.
- Vincular issues con commits y pull requests.

2.4 GitHub Actions (CI/CD)

GitHub Actions es la plataforma de automatización integrada en GitHub. Permite definir flujos de trabajo (workflows) que se ejecutan automáticamente ante eventos como un push o un pull request. Para el proyecto es útil para:

- Ejecutar pruebas automáticas del backend (Spring Boot o FastAPI).
- Construir y desplegar el frontend (React o Angular).
- Automatizar el cálculo y reporte de KPIs.

En febrero de 2026, GitHub Actions introdujo nuevas capacidades destacadas: autoscaling personalizado de runners sin necesidad de Kubernetes, controles de seguridad expandidos para todos los usuarios, y acceso anticipado a nuevas imágenes de Windows y macOS.

2.5 GitHub Copilot

GitHub Copilot es el asistente de inteligencia artificial integrado en la plataforma, disponible en el editor de código, en GitHub Mobile y en la interfaz web. Ayuda a los desarrolladores a escribir, completar, revisar y documentar código de forma más rápida. En 2026, el control remoto de sesiones de Copilot estará disponible de forma general en github.com y GitHub Mobile.

2.6 Seguridad: GitHub Advanced Security

GitHub ofrece funcionalidades de seguridad integradas que escanean el repositorio en busca de vulnerabilidades:

- Escaneo de código (Code Scanning): detecta vulnerabilidades en el código fuente.
- Escaneo de secretos (Secret Scanning): alerta cuando se expone una clave o token en el repositorio.
- Alertas de dependencias (Dependabot): notifica cuando una librería del proyecto tiene una vulnerabilidad conocida.

3. Planes y Costos

GitHub ofrece diferentes planes según el tamaño y necesidades del equipo. A continuación se presenta una comparativa de los planes más relevantes para el proyecto:

Plan	Precio	Para quien	Repositorios privados	Relevancia para el proyecto
Free	\$0 USD/mes	Individuos y proyectos open source	Ilimitados	Limitado: sin analytics avanzado
Teams	~\$4 USD/user/mes (~\$15,200 COP/user/mes)	Equipos pequeños y medianos	Ilimitados	Recomendado para el equipo de 6
Enterprise	~\$21 USD/user/mes	Empresas grandes	Ilimitados + controles avanzados	Opcional: si requieren compliance

4. Conceptos Clave para el Proyecto

El equipo debe familiarizarse con los siguientes términos antes de comenzar el desarrollo:

Concepto	Descripción	Aplicación en el proyecto
Repository (Repo)	Espacio donde se almacena el código y su historial	Un repo por cada componente: backend, frontend, laC
Branch	Rama de desarrollo independiente del código principal	Ramas por feature, por integrante o por sprint
Commit	Registro de un cambio con autor y descripción	Cada tarea completada genera uno o más commits
Pull Request (PR)	Propuesta de cambio para revisión del equipo	Obligatorio para fusionar cambios al branch principal
Issue	Tarea, bug o mejora registrada en el repositorio	Cada tarea semanal puede ser un issue asignado
GitHub Actions	Automatización de flujos de trabajo (CI/CD)	Build y deploy automatico del backend y frontend
API REST / GraphQL	Interfaces para consultar datos de GitHub programáticamente	Base técnica de la cuarta tarea semanal
Token de acceso	Credencial para autenticarse con la API de GitHub	Necesario para las pruebas de recuperación de información

5. Relevancia para los 5 Pilares del Proyecto

GitHub tiene impacto directo en los cinco pilares definidos en el Project Charter:

Pilar	Como GitHub lo soporta
1. Análisis y diseño de software	Issues y Projects para documentar decisiones de diseño y arquitectura
2. Desarrollo de software (Backend y Frontend)	Repositorios, branches y pull requests para todo el ciclo de desarrollo
3. Infraestructura como Código (IaC)	Repositorio dedicado para scripts de Docker y Terraform
4. Automatización de Tareas	GitHub Actions para CI/CD y calculo automatico de KPIs
5. Analisis de Datos	La API de GitHub es la fuente de datos principal para los reportes y visualizaciones

6. Conclusiones

GitHub es mucho más que una herramienta de almacenamiento de código. Para este proyecto representa la columna vertebral del desarrollo: gestiona el trabajo del equipo, automatiza los procesos y provee los datos que la aplicación necesita procesar.

Se recomienda al equipo:

- Crear la organización o repositorios del proyecto en GitHub Teams antes de la próxima semana.
- Configurar un Personal Access Token (PAT) o una GitHub App para las pruebas de la API (tarea 5 de esta semana).
- Definir la convención de branches y commits que usará el equipo durante los 5 meses del proyecto.
- Revisar el changelog de GitHub para estar al día con los cambios en la API, especialmente el nuevo formato de tokens de 2026.

Referencias

GitHub Docs: <https://docs.github.com>

GitHub Changelog: <https://github.blog/changelog/>

GitHub Blog - Product News: <https://github.blog/news-insights/product-news/>

GitHub Actions 2026 Roadmap:

<https://github.blog/changelog/2026-02-05-github-actions-early-february-2026-updates/>

TRM oficial 20/05/2026: \$3,796.87 COP/USD - Banco de la Republica de Colombia